

DDO STUDIO

TECHNICAL DOCUMENTATION

Architecture, DAT Asset Reconstruction, Skeletons, Materials, Textures,
Animation, and GLB Export

Version 1.7.2

Reverse-engineered implementation notes for the current DDO Studio codebase.

This is not an official Dungeons & Dragons Online file-format specification.

Purpose and Scope

This document explains how DDO Studio 1.7.2 turns data from a locally installed Dungeons & Dragons Online client into a focused searchable model library, textured and rigged 3D models, decoded animation data, and GLB files that can be viewed in the built-in Three.js viewer or opened in external tools such as Blender.

The emphasis is on the working reconstruction path: DAT loading, renderable-asset discovery, Setup and RenderMesh parsing, skeleton recovery, skin weights, materials and textures, Havok animation decoding, skeleton-family compatibility checks, curated animation naming, per-class viewer defaults, glTF/GLB construction, and the desktop/viewer architecture.

Scope note: Abandoned experiments and superseded UI/debug work are intentionally omitted. Where the project still has an unresolved reverse-engineering problem, the documentation calls it out explicitly rather than presenting a hypothesis as a solved format.

Current Capability Snapshot

Area	Status	Current behavior
DAT access	Stable	Loads the user's local DDO DAT set and exposes raw records through the local backend.
Asset index/search	Stable	Builds a schema-versioned LocalAppData model index from GameLogic properties. The public library includes directly renderable records plus standalone Weapons and Shields; unreliable worn/composed equipment remains outside normal search.
Mesh reconstruction	Stable	Setup -> RenderMesh -> vertex/index data -> glTF primitives.
Skeleton reconstruction	Stable for parsed Setups	Rebuilds joint names, hierarchy, local transforms, global bind transforms, and inverse-bind matrices.
Skinning	Stable for current tested formats	Recovers grouped DDO joint indices and weights and writes JOINTS_0 / WEIGHTS_0.
Diffuse/normal textures	Stable for DXT1/3/5	Resolves DDO material properties, decodes block-compressed surfaces, embeds PNGs in GLB.
Havok animation decode	Working with supported record types	Native decoder handles spline-compressed and interleaved/uncompressed animations.

General animation binding	Still incomplete	Track order is decoded, but a universal track-to-skeleton mapping is not proven for every skeleton family.
Animation naming/defaults	Curated + compatibility-gated	Ships a curated animation-name map with local user overrides. Compatible models prefer 0x05000440; Weapon previews prefer looping 0x05005943 and use weapon-specific viewer transforms.

Document Map

Section	What it covers
1-3	Application architecture, local services, DDO installation discovery, and DAT organization.
4-7	Asset indexing, model discovery, RenderMesh reconstruction, skeletons, and vertex skinning.
8-10	Materials, textures, UVs, coordinate conversion, and GLB construction.
11-14	Animation discovery, Havok decoding, family matching/binding, curated names, and default playback.
15-20	Viewer/Scene Studio, class-based transforms, export flow, runtime design, limits, source map, and glossary.

1. Application Architecture

DDO Studio is intentionally split into three cooperating pieces. The desktop application is the user-facing shell. A private local ASP.NET backend reads DDO DAT records and exposes structured/raw data over localhost. A separate exporter converts a selected Setup into a self-contained GLB. The embedded Three.js viewer then displays the exported GLB and provides scene, terrain, sky, animation, and presentation controls.

```

Windows desktop (WinForms + WebView2)
  |
  | launches / talks to localhost
  v
Local DdoDatApi backend <----> user's installed DDO DAT files
  |
  | raw/structured asset data
  v
DDOglbExporter -----> rigged/textured GLB

```

|
v
Three.js viewer inside WebView2

Component	Technology	Primary responsibility
DDOStudio.exe	.NET 10 WinForms + WebView2	Search/browse UI, DDO path selection, exporter orchestration, viewer host.
DdoDatApi	ASP.NET Core local service	DAT loading, property/index access, raw records, texture/animation diagnostics and decoding.
DDOGlbExporter	.NET console application	Reconstructs mesh, skeleton, skin, materials, textures, and optional validated animations into GLB.
Viewer	Three.js	Renders GLB, controls camera/model position and rotation, Scene Studio, terrain/skies, lighting, animation browsing, and presentation mode.

Important design choice: The application never needs to copy or redistribute DDO's DAT files. It reads the files from the user's own installation and writes only reconstructed/derived output.

2. Finding and Loading the DDO Data

At startup the backend resolves the DDO installation path in three steps: an explicit DDO_INSTALL_PATH environment variable, the Windows uninstall registry entry, and finally the common Program Files (x86) installation location. Once resolved, the backend opens the client DAT files through VoK.Sdk's DatFactory.

DAT group	Role in DDO Studio
client_gamelogic.dat	Properties, database objects, item/NPC metadata, names and search/index inputs.
client_general.dat	Setup objects, materials, render metadata, and many cross-referenced graphics records.
client_mesh.dat	RenderMesh geometry and vertex/index streams.
client_anim.dat	Animator/Havok animation records.
client_highres.dat / graphics records	High-resolution render surfaces used by textures when available.
client_surface.dat	Additional surface/graphics data exposed by the backend.
client_local_English.dat	Localized English resources.
cell/map/sound DATs	Loaded and available for broader asset exploration; not all are part of the model-export hot path.

Object IDs are not treated as arbitrary integers. VoK.Sdk exposes known ObjectType ID ranges. DDO Studio turns those ranges into an internal table so raw 32-bit references can be recognized as plausible DDO object references during relationship scanning.

3. The Local Backend and Raw DAT Access

The local backend is the bridge between the installed game data and the exporter/viewer. It binds only to the application workflow and exposes controllers for raw DAT reads, indexed game objects, textures, animations, relationships, and other asset classes.

```
RawDat/{dat}/{id} -> exact record bytes from the selected DAT
```

That raw-record endpoint is especially important because it keeps reverse engineering lossless. Higher-level parsers can fail or change without destroying the original record. The exporter and animation tools can always request the exact bytes again.

Generated caches are written beneath LocalAppData instead of Program Files. This matters for a normal Windows installation because standard users do not have write access to the install directory.

4. Asset Discovery and Search Indexing

DDO Studio's searchable library is built primarily from GameLogic database properties rather than by trying to name every binary graphics record directly. The index builder scans GameLogic objects in the DbProperties ID range, reads property collections through VoK.Sdk, derives the DDO Weenie type, and uses the SDK name generator to obtain human-readable names. The backend keeps internal equipment metadata, but publishes only model-oriented records to the desktop library: directly renderable non-equipables plus standalone Weapon and Shield records.

1. Enumerate GameLogic DAT records.
2. Keep records that fall inside the DbProperties object range.
3. Load the property collection through the DDO property master.
4. Determine the object's Weenie type and generate a human-readable name.
5. Store ID-to-name, name-to-ID, type groupings, internal equipment metadata, and the public model-name index in LocalAppData JSON caches.
6. Use the public model-name index for Search and View All. The desktop no longer exposes separate Equipment, Weapons, Armor, or similar category selectors.

Why this matters: The visible game/database record and the renderable model are often separate layers. Search begins with a named object, follows its visual/setup relationships, and only exposes entries that resolve to a nonzero Setup. Standalone Weapons and Shields stay searchable; worn/composed equipment can remain available to internal resolvers without cluttering the normal model browser.

5. From a Setup ID to Visible Geometry

The key renderable object for the exporter is a DDO Setup. A Setup is read from the General DAT and acts as the assembly point for a model. For visible geometry, the exporter reads the Setup's RenderMeshIds and fetches each corresponding RenderMesh from the Mesh DAT.

```
Setup (General DAT)
|
+--> RenderMeshIds[]
|
+--> RenderMesh (Mesh DAT)
|
+--> VertexArray / Vertices
+--> Indices / triangles
+--> MaterialDids[]
```

Each RenderMesh can contain multiple vertex sets, index groups, and material slots. DDO Studio converts each usable vertex/index group into a glTF primitive. Tiny helper meshes with only a handful of vertices are skipped because they do not represent visible model geometry.

DDO field	glTF output
Vertex.Position	POSITION accessor
Vertex.Normal	NORMAL accessor
UVMa[0]	TEXCOORD_0 accessor
Triangle a1/a2/a3	Index accessor, TRIANGLES mode
Recovered joint indices	JOINTS_0 accessor
Recovered weights	WEIGHTS_0 accessor
Material slot	glTF material reference

6. How DDO Studio Recovers Skeletons

The skeleton is recovered from the Setup itself. When VoK.Sdk parses a Setup, rigged models expose a HavokSetup member. That structure contains three pieces that proved sufficient to reconstruct the bind skeleton: BoneNames, ParentIndices, and a packed Bones array.

Core finding: Skeleton recovery does not infer bones from mesh shape. The joint list and hierarchy are present in the parsed Setup/Havok data.

6.1 Joint count and names

DDO Studio takes the usable joint count as the minimum of the bone-name count, parent-index count, and half of the packed Bones record count. The Bones array is interpreted as two records per joint.

```
jointCount = min(
    BoneNames.Count,
    ParentIndices.Count,
```

```
Bones.Count / 2
)
```

6.2 Local transform reconstruction

For each joint, the current implementation reads two packed Havok bone records, labeled here A and B. The following mapping is the empirically validated interpretation used by the exporter:

Joint component	Current interpretation
Translation	A.V1 (X, Y, Z)
Quaternion	(A.V2.Y, A.V2.Z, B.V1.X, B.V1.Y), normalized
Scale	(B.V1.Z, B.V2.X, B.V2.Y); all-zero scale becomes (1,1,1)
Name	BoneNames[i], or generated Bone_XX fallback

This mapping is implementation knowledge derived from the current DDO data and validated by coherent joint hierarchies, usable inverse-bind matrices, and correctly skinned test models. It should be treated as a DDO Studio decoder interpretation rather than a published Turbine/Standing Stone format definition.

6.3 Parent hierarchy

ParentIndices are encoded rather than stored as a plain zero-based parent index. The current rule treats raw value 3 or any odd value as a root/no-parent marker; otherwise the parent is raw/2. Invalid self-references and out-of-range parents are rejected back to root.

```
parent = (raw == 3 || raw is odd) ? -1 : raw / 2
```

6.4 Bind pose and inverse-bind matrices

Each joint's local scale, rotation, and translation are composed into a local matrix. Parent transforms are accumulated recursively to build global bind matrices. The exporter then inverts every global bind matrix and writes the result to glTF's inverseBindMatrices accessor.

```
Local = Scale * Rotation * Translation
Global[joint] = Local[joint] * Global[parent]
InverseBind[joint] = inverse(Global[joint])
```

The joint nodes, parent/child hierarchy, root joint, and inverse-bind matrices are then emitted as a standard glTF skin.

7. The Vertex Skinning Breakthrough

Recovering the skeleton was only half of the rigging problem. The mesh vertices also had to be connected to the correct joints with the correct weights. The important discovery was that the SDK's parsed representation of the vertex bone stream did not match the logical grouping used by the DDO data.

The vertex format stores the influence count in the high portion of VertexFormat. DDO Studio currently derives it with:

```
influences = (VertexFormat & 0xF0000) >> 16
```

VoK.Sdk exposed the bone stream as repeated records that looked like [byte prefix][float]. Reinterpreting those records literally produced bad joint assignments. The working interpretation is that the underlying bytes are grouped as all N joint-index bytes first, followed by all N float weights.

SDK-shaped view: [idx0][float0] [idx1][float1] [idx2][float2] ...

Actual logical layout: [idx0][idx1][idx2] ... [weight0][weight1][weight2] ...

7. Reconstruct the original $5 \cdot N$ bytes from the SDK's parsed prefix/float records.
8. Read the first N bytes as the actual logical DDO joint indices.
9. Read N floats beginning at byte offset N as the weights.
10. Clamp to glTF's four influences per vertex.
11. Reject out-of-range joint indices to joint 0.
12. Normalize the weights when their sum is non-zero; otherwise fall back to one full weight on joint 0.

Why it mattered: Treating InfluencedBones as a remapping palette collapsed valid limb indices into unrelated bones. Using the grouped index bytes directly preserved the actual joint numbers and produced coherent deformation.

8. Material and Texture Reconstruction

A RenderMesh does not directly contain a finished PNG/JPEG texture. Instead, its material slot references a chain of DDO render objects. DDO Studio walks that chain and converts the texture surface into an ordinary embedded PNG for glTF.

```
RenderMesh.MaterialDids[]
|
v
MaterialInstance
|
v
MaterialModifier(s)
|
v
MaterialProperty (name + TextureDid)
|
v
RenderTexture
|
v
RenderSurface candidate(s)
|
v
DXT1 / DXT3 / DXT5 payload
|
v
RGBA -> PNG -> embedded GLB image
```

8.1 Finding the diffuse and normal maps

The exporter parses each MaterialInstance and follows its Modifiers. Every MaterialModifier can expose MaterialProperties containing a MaterialPropertyName and TextureDid. Later modifiers are allowed to overwrite earlier values, matching the idea of layered/overridden material state.

Recognized semantic	Names currently treated as equivalents
Diffuse/base color	DiffuseMap, BaseColorMap, ColorMap, AlbedoMap
Normal map	NormalMap, NormalTexture, BumpMap

If no named diffuse property is found but at least one texture ID exists, the first discovered texture is kept as a diffuse fallback. Other named DDO texture properties are preserved in glTF material extras so the information is not thrown away even when no PBR mapping exists yet.

8.2 Resolving RenderTexture to RenderSurface

A RenderTexture contains a list of SurfaceDids. In the tested DDO records, the list is generally ordered with the highest-resolution surface first. DDO Studio checks candidate surfaces from the graphics DAT sources and prefers the first surface with a pixel format the exporter can decode.

8.3 RenderSurface header and block decompression

The working RenderSurface interpretation used by the exporter reads width, height, FourCC, and payload size from the record header. Supported compressed payloads are DXT1, DXT3, and DXT5 (BC1/BC2/BC3-style block compression).

Header offset	Meaning
+8	uint32 width
+12	uint32 height
+16	4-byte ASCII FourCC
+20	uint32 compressed payload length
+24	start of compressed texture payload

The exporter contains its own DXT block decoders. It expands the compressed blocks into RGBA pixels, writes a PNG in memory, and embeds that PNG directly into the GLB's BIN chunk. This keeps the exported model self-contained.

Current texture limit: The GLB exporter intentionally supports the proven DXT1/DXT3/DXT5 path. Other DDO surface encodings can still be discovered and preserved as IDs, but require additional decoders before they can become glTF textures.

9. UVs, Normals, and Coordinate Conventions

DDO vertices can contain multiple UV entries. DDO Studio records diagnostic information about all channels but currently exports UV channel 0 as TEXCOORD_0. A key finding from texture-wrap tests was

that DDO's raw V coordinate should not be globally flipped. Earlier blanket V flipping caused otherwise correct textures to map incorrectly.

Normals are normalized when present. Missing normals fall back to +Z for safety.

9.1 DDO Z-up to glTF Y-up

DDO model coordinates are Z-up while glTF expects Y-up. Rather than rotating every vertex and every joint independently, the exporter creates one common scene-root conversion node with a -90 degree X rotation. Both the skeleton and skinned mesh nodes live beneath that same conversion node, preserving their bind relationship.

```
DDO roots (skeleton + meshes)
  |
  v
DDO_Zup_to_glTF_Yup [rotation = -90° around X]
  |
  v
glTF scene
```

10. Building the GLB

DDO Studio writes glTF 2.0 directly and packages the JSON and binary payload into a GLB. The binary buffer contains vertex attributes, indices, inverse-bind matrices, animation samples, and embedded PNG bytes.

glTF feature	DDO Studio source
Meshes/primitives	RenderMesh vertex sets and index groups
Skin	Setup.HavokSetup skeleton + inverse-bind matrices
JOINTS_0 / WEIGHTS_0	Recovered DDO vertex influence stream
Base color texture	Diffuse/base-color material property
Normal texture	Normal-map material property
Animations	Decoded and validated transform tracks when considered compatible
Extras	Additional discovered DDO texture IDs not yet mapped to standard PBR slots

Materials default to OPAQUE. Earlier experiments with BLEND made otherwise correct DDO materials appear ghosted, so opacity is not enabled unless the material pipeline later gains reliable alpha semantics.

11. Animation Discovery

Animations live in client_anim.dat, but identifying which anonymous 0x05... records belong to a selected model is separate from decoding their transform tracks. DDO Studio therefore separates discovery evidence from playback certainty.

The animation catalog enumerates Anim DAT IDs, scans records for Havok class strings/chunk signatures, and can expose exact raw records for inspection. For a selected Setup, conservative candidate discovery looks for direct Setup-to-animation references and reverse animation-to-Setup references.

Safety rule: A relationship candidate is not automatically labeled as playable. The UI is designed to keep unresolved DAT records distinct from successfully decoded/compatible animation clips.

Relationship tools also follow exact aligned DDO object references through small structured metadata records such as animator/state/script records, while excluding giant material/texture/mesh blobs as relationship seeds. This reduces false positives from coincidental 32-bit values.

12. Native Havok Animation Decoding

DDO Studio now contains a native decoder for the older Havok binary-tag animation records found in DDO's Anim DAT. It does not require external Havok SDK binaries or a separate conversion utility.

Supported class/path	Current support
hkaSplineCompressedAnimation	Decoded to per-frame translation, quaternion rotation, and scale tracks.
hkaInterleavedUncompressedAnimation	Decoded as hkQsTransform records (12 floats / 48 bytes per track per frame).
Spline scalar quantization	8-bit and 16-bit
Spline rotation quantization	POLAR32, THREECOMP40, THREECOMP48, UNCOMPRESSED
Spline degree	0 through 3

DDO wraps the Havok binary tagfile in a small record header, so the decoder does not assume that Havok starts at a fixed byte offset. It searches for the binary-tag magic pair and then locates/validates the animation object within the record.

Havok magic bytes searched by the decoder: CAB00D1E / D011FACE

On a successful decode, the result contains clip duration, transform-track count, frame/block metadata, track summaries, and optionally a frame-by-frame list of transforms. On failure, the raw DAT record remains available, which is important because unsupported variants can be added later without losing source data.

13. Skeleton Family Matching and the Remaining Binding Problem

Decoding an animation answers 'what does track 0, track 1, track 2... do over time?' It does not automatically prove that track N belongs to joint N on every DDO skeleton. The generalized track-to-joint binding layer remains the main unresolved animation problem.

13.1 Evidence used today

- The selected Setup's parsed joint count.
- Animator BoneCount from Anim DAT records.
- The decoded transform-track count.
- Relationship evidence through Setup/state/script/animator records.
- Pose-family tests that compare relatively stable decoded translations against the Setup bind translations.
- Successful playback/export validation for known families.

The family matcher first requires equal counts, then samples up to 64 animation frames. For tracks whose translation remains sufficiently stable, it compares the median decoded translation with the corresponding bind-pose joint translation. A high proportion of close matches is evidence that the clip and Setup probably share a skeleton family; strong mismatches can filter out same-count but wrong-family clips.

Important limitation: Equal bone count is evidence, not ownership proof. Two unrelated skeletons can have the same number of joints. DDO Studio deliberately keeps this distinction in the compatibility logic.

13.2 Conservative GLB animation embedding

When the exporter is explicitly given an animation to embed, it currently requires the decoded transform-track count to equal the target joint count, requires frame data, and requires track indices to arrive in exact 0..N-1 order. Only then are translation/rotation/scale channels written to the glTF joint nodes.

This conservative behavior prevents an apparently successful export from silently animating the wrong bones.

14. Curated Animation Names and Default Playback

DDO Studio 1.7.2 uses a curated built-in animation-name map stored with the viewer as viewer/animation-names.json. A clean installation therefore shows known labels without requiring a user-specific alias file. Runtime renames are still supported through %LOCALAPPDATA%\DDO Asset Studio\animation-aliases.json; local aliases override the built-in name for the same animation ID and are never bundled into release packages.

Rule	Current behavior
Built-in names	viewer/animation-names.json supplies curated labels in a clean install.
Local rename override	%LOCALAPPDATA%\DDO Asset Studio\animation-aliases.json overrides a built-in label for the same ID.
General default animation	If compatible, 0x05000440 is selected as the normal model default.
Weapon default animation	If compatible, Weapon previews select

	0x05005943 and enable looping. If unavailable, compatible 0x05000440 remains the fallback.
Weapon Position	X = 0.00x, Y = 1.40x, Z = 0.00x.
Weapon Rotation	X = 90 degrees, Y = 0 degrees, Z = 0 degrees.
Reset behavior	Reset Model Transform restores the default transform for the current model classification.
Pedestal separation	Model transforms never translate or rotate the pedestal; pedestal height and scale are independent controls.

Default playback is compatibility-gated. The program never forces a preferred animation that is absent from the current Setup's compatible list. Weapon defaults are classification-based, so they apply to Weapon WeenieType 0x00020081 generally rather than to a particular database object.

Design rule: Curated names and presentation defaults improve browsing without changing the underlying DAT data. Animation defaults are used only after skeleton compatibility has been established, and per-class model transforms are viewer presentation state rather than modifications to exported geometry.

15. Viewer and Scene Studio

The desktop viewer is hosted in WebView2 and rendered with Three.js. Models are displayed from the GLB generated by the exporter rather than by reimplementing DDO rendering directly in JavaScript.

Scene Studio adds environment presentation without changing the underlying model data. DDO Studio 1.7.2 ships with no terrain, HDR sky, cubemap, or other environment artwork. Users can import their own equirectangular HDR/standard-image skies, six-face cubemaps, GLB terrain, and self-contained GLTF terrain for the current session. Environment files are opened locally and are not uploaded by DDO Studio.

Environment packaging change: the installer contains viewer code and runtime libraries only. It does not contain user terrain or sky assets. Because browser security prevents silent reopening of arbitrary local files, imported environment files are selected again after an application restart.

15.1 Model transforms and class defaults

The model is placed beneath its own Three.js transform root and exposes Position X/Y/Z plus Rotation X/Y/Z. Most models begin at zero translation and zero rotation. Weapon records (WeenieType 0x00020081) use a presentation default of Position Y = 1.40x and Rotation X = 90 degrees, with every other position/rotation axis at zero. Reset Model Transform restores those class defaults. The pedestal is a separate scene root: changing model Position Y or any rotation does not move the pedestal. Pedestal height and scale are controlled independently.

15.2 Terrain placement

Imported terrain uses its own transform and uniform scale. On load, its bounding box is measured, grounded, and composed behind the selected model relative to the reset-camera viewing direction. This camera-relative placement avoids both the older overlap problem and the diagonal-view bug where the model could appear at the far left or right edge of a large environment. X, Y, Z, rotation, scale, visibility, and clear controls remain available for manual correction.

15.3 Presentation mode

Presentation/View Only mode hides desktop chrome and viewer controls while preserving the current model, camera, animation, user-imported environment, terrain, and lighting state for the current session. It is intended for clean model viewing rather than diagnostics.

16. End-to-End Model Export Flow

13. The user searches for or selects an asset in the desktop application.
14. DDO Studio resolves the renderable Setup ID for that asset.
15. The desktop launches the exporter for the selected Setup.
16. The exporter asks the local backend for the Setup record from General DAT.
17. HavokSetup is decoded into a skeleton when present.
18. RenderMesh IDs are followed into Mesh DAT and converted into glTF primitives.
19. Per-vertex DDO joint indices and weights are reconstructed and written as glTF skinning attributes.
20. Material IDs are followed through MaterialInstance -> MaterialModifier -> texture properties.
21. RenderTexture surface candidates are decoded from DXT into PNG and embedded in the GLB.
22. The skeleton and mesh are placed under the common DDO Z-up -> glTF Y-up conversion root.
23. If explicitly validated animations are requested, decoded transform channels are embedded conservatively.
24. The completed GLB is loaded by Three.js for preview or written as a permanent export. For preview, the desktop passes classification-based model-transform defaults to the viewer, loads the compatible animation catalog, applies curated animation names, and starts the preferred compatible default animation when one is defined.

```
Named game object
-> public model-index entry
-> visual/setup relationship
-> Setup
  -> HavokSetup -> skeleton -> skin
  -> RenderMeshIds -> geometry + skin weights
  -> MaterialDids -> texture chain -> PNG
  -> optional validated Havok clips
-> GLB
-> Three.js preview
  -> class-based Position/Rotation defaults
  -> compatible animation list + curated names
-> Blender / downstream conversion (including STL workflows)
```

17. Windows Runtime and Installation Design

The release build publishes the .NET applications for win-x64 as self-contained executables. The installer also carries the offline WebView2 Evergreen Standalone prerequisite. The goal is that a fresh Windows user installs DDO Studio without separately installing .NET, Node.js, build tools, or WebView2.

The end user still needs a local DDO installation because DDO Studio deliberately does not package the game's DAT files. On first use, the application can detect the standard DDO installation or accept a folder selected by the user.

Search/index caches, transient preview files, diagnostics, WebView2 state, and local animation overrides live in the current Windows user's profile rather than beside the executable in Program Files.

Release/source packages intentionally exclude those runtime files and PDB debug symbols so machine-specific paths and user-specific state are not distributed.

18. Known Limits and Reverse-Engineering Boundaries

- General track-to-joint animation binding is not proven for every skeleton family.
- Texture decode currently focuses on DXT1/DXT3/DXT5 surfaces; other formats require additional decoders.
- Only the first UV channel is exported as TEXCOORD_0 even though DDO vertices can contain multiple UV channels.
- Only diffuse/base-color and normal maps are mapped to standard glTF PBR slots today. Other discovered texture roles are retained in material extras.
- Some DDO assets contain helper or unusual mesh records that are intentionally skipped or require additional format-specific handling.
- Curated animation names cover only IDs that have been manually identified. Unknown IDs remain anonymous rather than receiving a fabricated developer name.
- The documented packed skeleton/skin interpretations are the current DDO Studio implementation and are empirical rather than an official published game specification.

Engineering principle: When DDO Studio cannot prove an interpretation, it prefers preserving raw data and surfacing uncertainty over silently manufacturing a 'working' result.

19. Source Map for Maintainers

File	Why it matters
src/DdoDatApi/DatSource.cs	Locates DDO, loads DAT files, property master, image/sound exporters, and ObjectType ID ranges.
src/DdoDatApi/Caching/IndexLoader.cs	Builds and loads searchable GameLogic/property caches.
src/DdoDatApi/Controllers/RawDatController.cs	Lossless raw-record access used by exporter/research tools.
src/DDOExporter/Program.cs	Model reconstruction: Setup, skeleton, skinning, RenderMesh, materials, DXT decode, GLB writer.
src/DdoDatApi/Controllers/DdoHavokAnimationDecoder.cs	Native Havok spline/interleaved animation decoder.
src/DdoDatApi/Controllers/AnimationCatalogController.cs	Anim DAT enumeration, Havok signatures, raw inspection,

	relationship candidates.
src/DdoDatApi/Controllers/AnimationSkeletonMatcherController.cs	Joint-count/skeleton-family filtering and empirical pose matching.
src/DDOAssetStudio/viewer/animation-names.json	Curated built-in animation labels shipped with the viewer; local aliases can override them at runtime.
src/DDOAssetStudio/Program.cs	WinForms shell, backend/exporter orchestration, WebView2 host, asset browsing/export UI.
src/DDOAssetStudio/viewer/index.html	Three.js viewer, Scene Studio, XYZ position/rotation controls, class defaults, terrain/skies, animation browser, and presentation mode.
SCENE_STUDIO.md	Current environment import, terrain placement, independent pedestal behavior, and presentation controls.
docs/TECHNICAL_ARCHITECTURE.md	Concise current architecture: process layout, search boundary, model resolution, viewer transforms, animation defaults, and local storage.

20. Glossary

Term	Meaning in DDO Studio
DAT	DDO's client archive/data files containing game database records, graphics, meshes, animations, sound, maps, and related resources.
DID / object ID	32-bit DDO object identifier used to cross-reference records.
DbProperties	Property-driven game/database object layer used heavily for names and searchable gameplay objects.
Setup	Renderable model assembly record containing RenderMesh references and, for rigged assets, HavokSetup skeleton data.
RenderMesh	Geometry record containing vertex sets, indices, material slots, UVs, normals, and skin influence data.
HavokSetup	Parsed Setup substructure containing the bind skeleton information used by DDO Studio.
MaterialInstance	Material record that references modifiers.
MaterialModifier	Material override/property container that can name texture roles and point to RenderTexture IDs.
RenderTexture	Texture metadata record containing one or more RenderSurface references.
RenderSurface	Pixel/surface record containing dimensions, compression

	FourCC, and compressed image payload.
GLB	Binary glTF 2.0 file containing scene JSON plus binary buffers/images in one file.
Inverse-bind matrix	Matrix used by glTF skinning to transform a joint from model bind space during deformation.
Animation track	One decoded stream of translation/rotation/scale values over time; not automatically the same thing as a proven named skeleton joint.

Closing Summary

DDO Studio's model pipeline works because the DDO asset graph is reconstructed rather than flattened into one guessed blob. A named GameLogic object leads to a renderable Setup; the Setup leads to RenderMeshes and Havok skeleton data; the meshes lead to material records and vertex skinning; the material records lead to texture surfaces; and the exporter converts those separate pieces into a conventional glTF scene.

The largest remaining technical frontier is animation binding across every skeleton family. The raw animation data is present, the native Havok decoder can already recover frame transforms for supported record types, and skeleton-family filtering can reject many incompatible clips. The final universal step is proving how every decoded track index maps onto every DDO skeleton's joint ordering without relying on coincidence.

Documentation status: This manual describes the DDO Studio 1.7.2 final source implementation. It documents the active architecture and verified reconstruction paths, while intentionally omitting discarded one-off experiments and user-specific diagnostic history.